

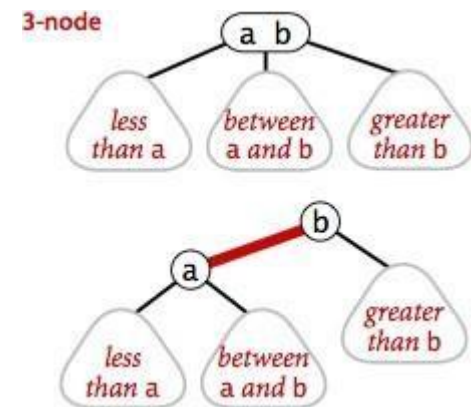
ALBERI ROSSO-NERI

IMPLEMENTAZIONE

Giovedì 26 Marzo 2026

RED-BLACK BST

- Ovvero, codifica degli alberi 2-3 a partire da BST, mediante aggiunta di informazione per codificare i 3-nodi, i quali sono rappresentati come coppie di 2-nodi, ciascuna coppia tenuta insieme da un arco rosso.
- I nodi rossi sono orientati a sinistra;
- Nessun nodo ha due archi rossi ad esso connessi;
- L'albero ha bilanciamento nero perfetto: ogni cammino dalla radice al link nullo ha lo stesso numero di link neri



```
public class RedBlackBST<K extends Comparable<K>, Value> {  
    private static final boolean RED = true;  
    private static final boolean BLACK = false;  
    private Node root; // root of the BST  
    private int n; // number of nodes  
    // BST node data type  
    private class Node {  
        private K key; // key
```



```
private Value val;    // associated data

private Node left, right; // links to left and right subtrees

private boolean color; // color of parent link

public Node(K key, Value val, boolean color) {

    this.key = key;

    this.val = val;

    this.color = color;

}

}
```



Ricerca in Java

```
public Value get(K key) {  
    return get(root, key);  
}  
  
public Value get(Node x, K key) {  
    while (x != null) {  
        int cmp = key.compareTo(x.key);  
        if (cmp < 0) x = x.left;  
        else if (cmp > 0) x = x.right;  
        else return x.val;  
    }  
}
```

```
}  
  
return null;  
}  
  
// ricerca  
  
public boolean contains(K key) {  
    return get(key) != null;  
}
```



Inserimento in Java

- + I casi si riducono alle seguenti istruzioni:
 - + Figlio destro rosso -> ruota a sinistra;
 - + Figlio sinistro rosso, nipote sinistro-sinistro rosso -> ruota a destra;
 - + Figli entrambi rossi -> commuta i colori.



Inserimento in Java

```
public void put(K key, Value val) {  
    root = insert(root, key, val);  
    root.color = BLACK;  
}  
  
private Node insert(Node h, K key, Value val) {  
    if (h == null) {  
        n++;  
        return new Node(key, val, RED);  
    }  
  
    int cmp = key.compareTo(h.key);  
    if (cmp < 0) h.left = insert(h.left, key, val);
```

```
        else if (cmp > 0) h.right = insert(h.right, key, val);  
        else h.val = val;  
  
        // corregge link rossi che tendono a destra  
        if (isRed(h.right) && !isRed(h.left)) h =  
            rotateLeft(h);  
        if (isRed(h.left) && isRed(h.left.left)) h =  
            rotateRight(h);  
        if (isRed(h.left) && isRed(h.right)) flipColors(h);  
        return h;  
    }  
}
```



```
public int size() {
```

```
    return n;
```

```
}
```

```
public boolean isEmpty() {
```

```
    return n == 0;
```

```
}
```

```
// height of tree
```

```
public int height() { return height(root); }
```

```
private int height(Node x) {
```

```
    if (x == null) return -1;
```

```
    return 1 + Math.max(height(x.left), height(x.right));
```

```
}
```

```
// return the smallest key; null if no such key
```

```
public K min() { return min(root); }
```

```
private K min(Node x) {
```

```
    K key = null;
```

```
    while (x != null) {
```

```
        key = x.key;
```

```
        x = x.left;
```

```
    }
```

```
    return key;
```

```
}
```




```
public int size() {
```

```
    return n;
```

```
}
```

```
public boolean isEmpty() {
```

```
    return n == 0;
```

```
}
```

```
// height of tree
```

```
public int height() { return height(root); }
```

```
private int height(Node x) {
```

```
    if (x == null) return -1;
```

```
    return 1 + Math.max(height(x.left), height(x.right));
```

```
}
```

```
// return the smallest key; null if no such key
```

```
public K min() { return min(root); }
```

```
private K min(Node x) {
```

```
    K key = null;
```

```
    while (x != null) {
```

```
        key = x.key;
```

```
        x = x.left;
```

```
    }
```

```
    return key;
```

```
}
```



```
// return the largest key; null if no such key
```

```
public K max() { return max(root); }
```

```
private K max(Node x) {
```

```
    K key = null;
```

```
    while (x != null) {
```

```
        key = x.key;
```

```
        x = x.right;
```

```
    }
```

```
    return key;
```

```
}
```



```
public static void main(String[] args) {  
  
    String test = "S E A R C H E X A M P L E";  
  
    String[] keys = test.split(" ");  
  
    RedBlackBST<String, Integer> st = new RedBlackBST<String, Integer>();  
  
    for (int i = 0; i < keys.length; i++) st.put(keys[i], i);  
  
    StdOut.println("size = " + st.size());  
  
    StdOut.println("min = " + st.min());  
  
    StdOut.println("max = " + st.max());  
  
    StdOut.println(); StdOut.println();  
  
}
```



Esercizio

- + *Scrivere un programma conta la percentuale di nodi rossi in un dato RedBlack BST. Testare il programma inserendo N chiavi random in un albero inizialmente vuoto, per $N = 10^4$, 10^5 , and 10^6 . Di ciascun albero valutare l'altezza.*
- + Riprodurre l'esperimento inserendo le chiavi in ordine crescente e decrescente



- + Aggiungere un metodo `is23()` che controlla che l'albero sia 2-3, ovvero che nessun nodo è connesso a due link rossi e non ci sono link rossi che vanno a destra.
- + Aggiungere un metodo `isBalanced()` per controllare che tutti i cammini dalla radice ai link null abbiano lo stesso numero di link neri.
- + Aggiungere un metodo `isBST()` che testa se è un albero binario di ricerca.
- + Combinare i metodi precedenti per creare il metodo `isRedBlackBST()`.

